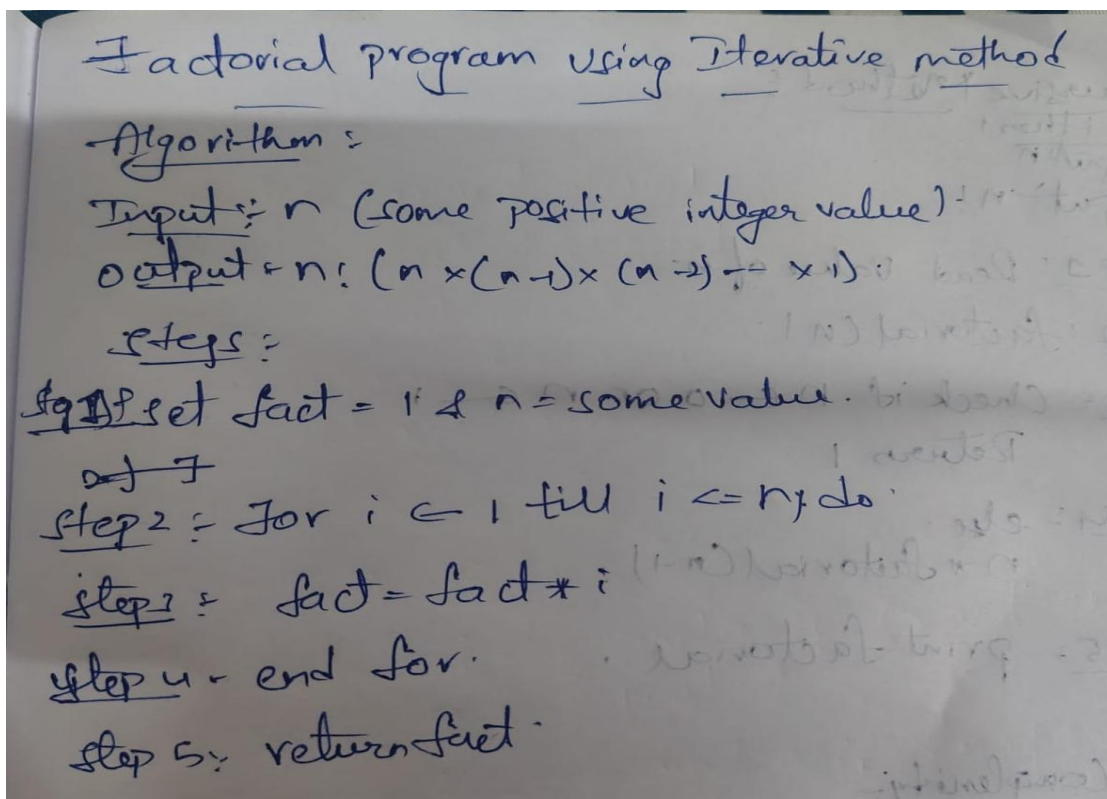


PRACTICAL- 4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

ITERATIVE METHOD

ALGORITHM:

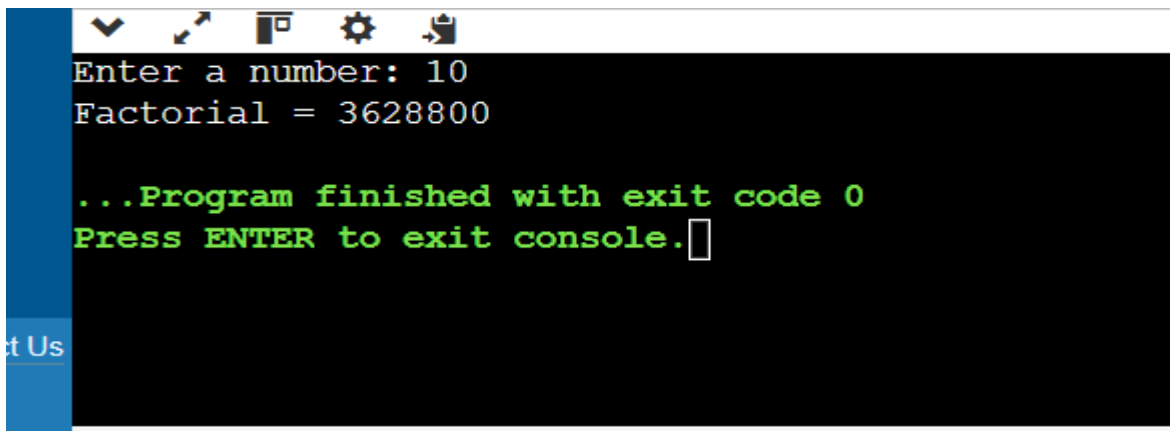


CODE:

```
#include <iostream>
using namespace std;
int factorial(int n)
{
    int fact = 1;
    for(int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    return fact;
}
int main()
```

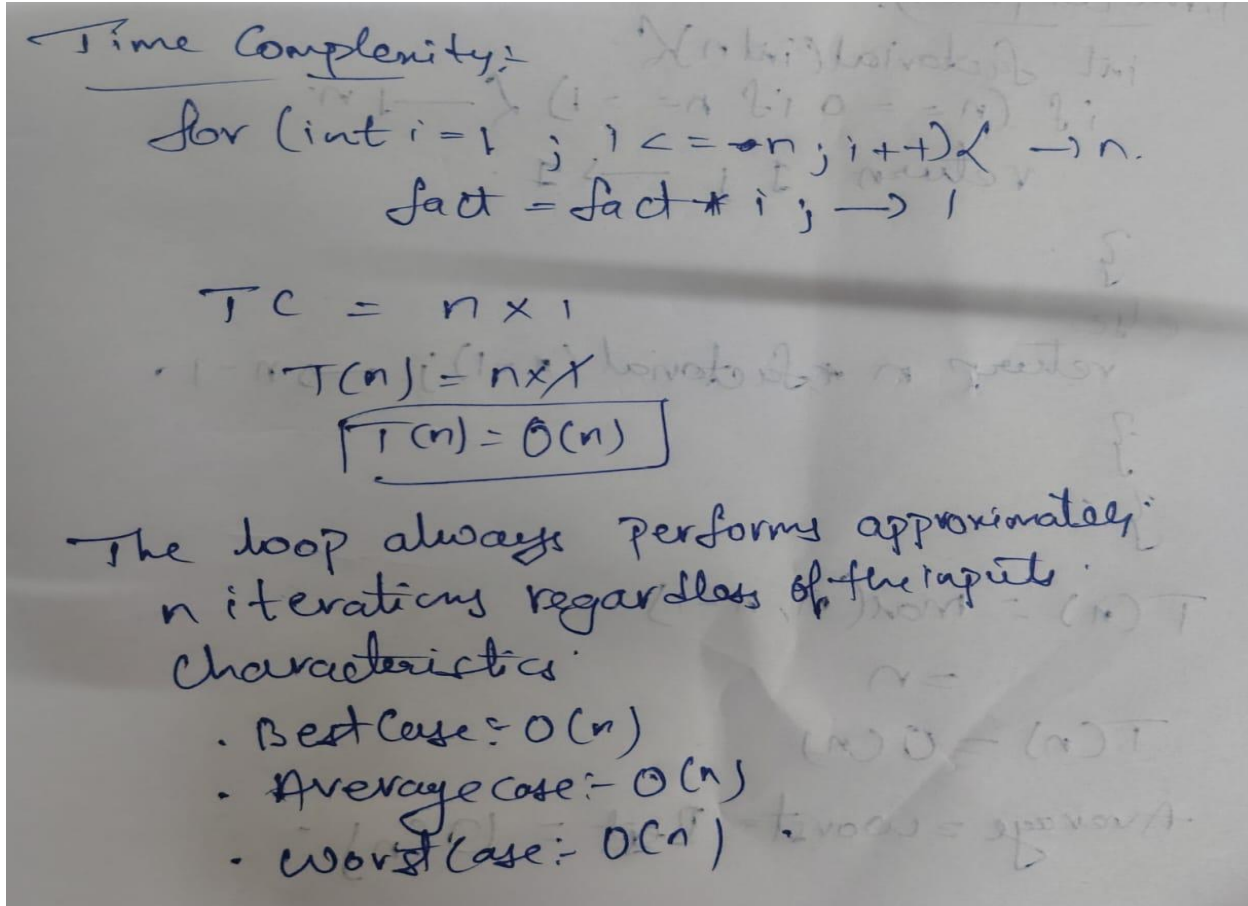
```
{  
int n;  
cout << "Enter a number: ";  
cin >> n;  
cout << "Factorial = " << factorial(n);  
return 0;  
}
```

OUTPUT:



```
Enter a number: 10  
Factorial = 3628800  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY:



Time Complexity:

```
for (int i = 1; i <= n; i++)  
    fact = fact * i;
```

$TC = n \times 1$

$T(n) = n \times 1$

$T(n) = O(n)$

The loop always performs approximately n iterations regardless of the inputs.

Characteristics:

- Best Case: $O(n)$
- Average case: $O(n)$
- Worst Case: $O(n)$

RECURSIVE METHOD

ALGORITHM:

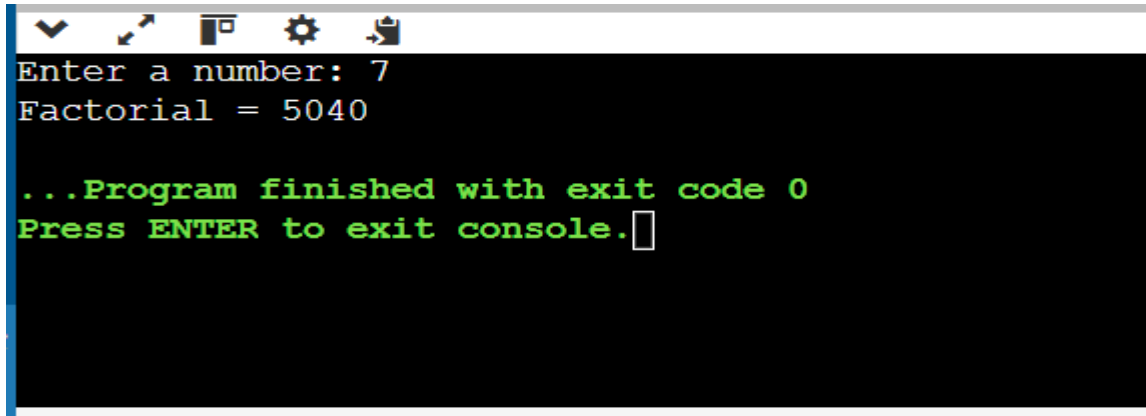
Recursive Method :-
Algorithm +
Input :- n
Output :- n! (factorial of n)
Step 1 :- Read value of n
Step 2 :- factorial(n)
Step 3 :- Check if $n \leq 0$ or $n \leq 1$:-
Returns 1
Step 4 :- else
 $n \times \text{factorial}(n-1)$
Step 5 :- print factorial

CODE:

```
#include <iostream>
using namespace std;
int factorial(int n)
{
    if(n == 0 || n == 1)
    {
        return 1;
    }
    return n * factorial(n - 1);
}
int main()
```

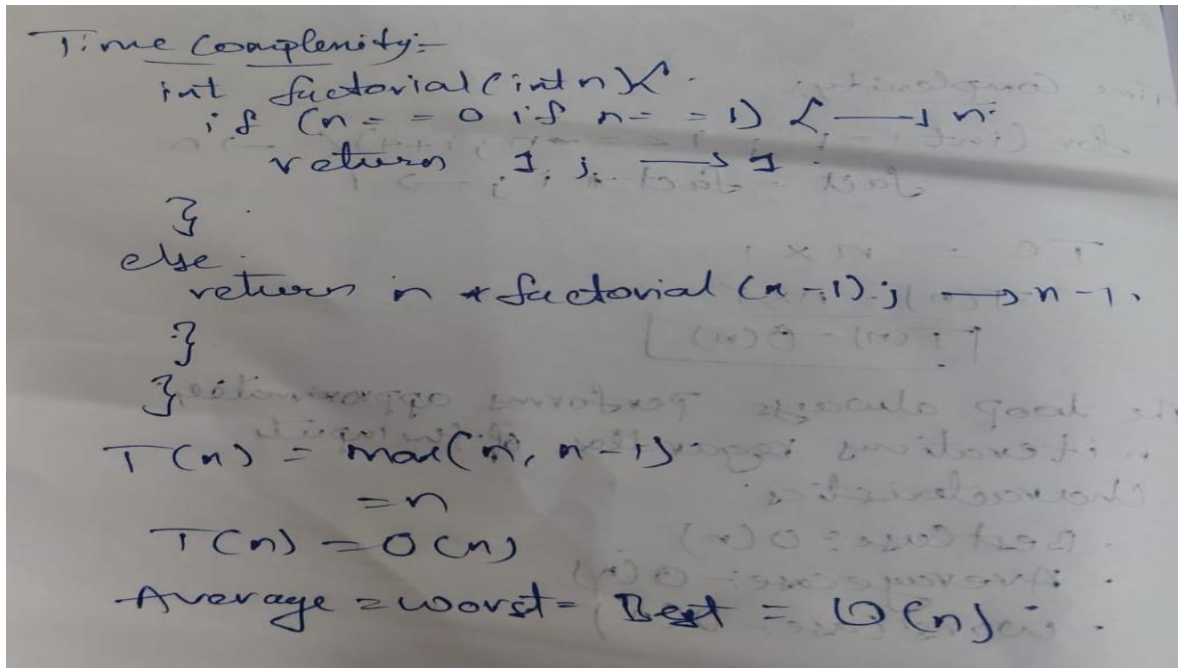
```
{  
int n;  
cout << "Enter a number: ";  
cin >> n;  
cout << "Factorial = " << factorial(n);  
return 0;  
}
```

OUTPUT:



```
Enter a number: 7  
Factorial = 5040  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY:



Time Complexity:-

```
int factorial(int n) {  
    if (n == 0 || n == 1) return 1;  
    return n * factorial(n-1);  
}
```

$T(n) = \max(n, n-1)$

$T(n) = O(n)$

Average = Worst = Best = $O(n)$

Department of Computer Engineering
01AI0506 Design and Analysis of Algorithm Lab Manual

CONCLUSION: The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.